Preparcial: mesa de reparaciones de dispositivos

Un taller técnico recibe dispositivos de clientes y registra órdenes de reparación. El proyecto carga un CSV de reparaciones, descarta órdenes canceladas, conserva en memoria las reparaciones válidas y calcula el importe estimado de cada trabajo.

Este proyecto se entrega aproximadamente un día antes del parcial. No hay tareas de implementación para esta instancia: el objetivo es ejecutar, leer y comprender el código que luego deberá evolucionarse.

Ejecución

Requiere JDK 21 o 25 y Maven 3.9.x. Desde la raíz:

```
mvn test
java -cp target/classes utnfc.backend.parcial.Main
java -cp target/classes utnfc.backend.parcial.Main datos/datos.csv
```

Luego de resolver dependencias puede probarse `mvn -o test`. No se requiere ninguna dependencia externa durante la ejecución.

Contrato del CSV

El archivo utiliza UTF-8 y comienza exactamente con:

```
id,cliente,dispositivo,marca,prioridad,horas,estado,costoBase
```

Cada registro tiene ocho campos separados por comas. No hay campos entrecomillados ni comas internas; se trata de un CSV controlado. Se quitan espacios externos y se conservan campos vacíos.

Los campos representan:

- `id`: identificador de la orden dentro del taller.
- `cliente`: persona titular del dispositivo. No se usa para agrupar ni calcular, pero permite saber a quién corresponde la orden.
- `dispositivo`: tipo de equipo reparado, por ejemplo `celular`, `notebook`, `tablet` o `consola`. El taller lo utiliza para obtener estadísticas por tipo de equipo.
- `marca`: fabricante del equipo. Es diferente de `dispositivo`: dos órdenes pueden ser de celulares de marcas distintas, y la marca permite obtener otra agrupación independiente.
- `prioridad`: nivel de atención solicitado, entre 1 y 3. A mayor prioridad, mayor cargo operativo.
- `horas`: estimación de horas de trabajo técnico.
- `estado`: situación de la orden dentro del taller.
- `costoBase`: importe fijo asociado al diagnóstico, recepción o preparación del trabajo.

Reglas del dominio

- `id`, `cliente`, `dispositivo`, `marca` y `estado` no pueden estar vacíos.
- `prioridad` debe ser un entero entre 1 y 3 inclusive.
- `horas` debe ser mayor que cero.
- `costoBase` no puede ser negativo.

- El importe estimado suma tres conceptos:
  1. `costoBase`, por la recepción y el diagnóstico inicial.
  2. `horas * 850`, por la mano de obra técnica estimada.
  3. `prioridad * 300`, por la organización operativa necesaria para atender el nivel de prioridad.

La fórmula es:

```
costoBase + horas * 850 + prioridad * 300
```

El objeto `Reparacion` debe proteger estas invariantes y concentrar las validaciones propias del dominio. La interpretación de los campos puede estar cerca del dominio mediante `desdeCampos` o una fábrica equivalente.

Estados y errores

- `CANCELADA` se descarta y no crea una reparación.
- `ABIERTA` y `LISTA` se procesan si los datos son válidos. `LISTA` representa una reparación cuyo diagnóstico ya fue realizado y que está lista para continuar en el taller.
- Cualquier otro estado es inválido en este modelo inicial.
- Una cantidad incorrecta de columnas, un número inválido, un campo vacío o un valor fuera de rango vuelve inválida la fila.
- El parser registra línea y motivo, y continúa procesando el archivo.
- Un encabezado incorrecto aborta con `IllegalArgumentException`.
- Un fallo de lectura propaga `IOException`.

El encabezado no cuenta como fila leída y debe cumplirse:

```
leídas = procesadas + descartadas + inválidas
objetos = procesadas
```

Recorrido sugerido

1. Ejecutar los tests y el `Main`.
2. Leer `Reparacion`, sus invariantes y `importe()`.
3. Seguir `ParserReparaciones` y `ResultadoParseo`.
4. Leer `Taller`, su copia defensiva, filtros, facturación total y agrupamientos por marca.
5. Localizar en el dataset casos normales, cancelados, límites e inválidos.

El dataset contiene exactamente 60 filas de datos más un encabezado. Los tests de dominio, parser y colección sirven como referencia del comportamiento esperado. El `Main` debe permitir observar las cantidades del procesamiento, la facturación y la agrupación por marca. Los getters y la clase de resultado usan Java convencional; no se agrega Lombok.